

Field Programmable Gate Array(FPGA): An Innovation In Hardware Technology

Farhanaaz

Assistant Professor

School of Computer Science and Applications
REVA University, Bangalore, India
Email: farhanaaz@reva.edu.in

Sanju V

Professor

Department of Computer Science and Engineering
REVA University, Bangalore India
Email: sanju.v@reva.edu.in

Abstract—FPGA is playing a significant role in the field of computer architecture and design. Other peer technologies of FPGA are embedded microprocessor and microcontroller. In this paper we are comparing the technologies which adapts to network-on-chip architecture. For embedded microprocessor and microcontroller, the hardware is already designed and we write the software instruction by exploiting the hardware. The logic or program developed for the same is purely software. To fetch each instruction, to execute, and to access data from external sources time taken is usually high. The nature of execution is sequential. But in FPGA we write the code to implement the hardware itself using HDL(Hardware Description language). HDL directly implements logic functions and interconnections. Design can be programmed and tested immediately even before fabricating on the chip. The nature of execution in HDL is highly parallel.

Index Terms—FPGA, Embedded microprocessor, Microcontroller, Programming languages, Hardware Description Language

I. INTRODUCTION

With the emergence of technology in every area of human life, we look for more sophisticated technological advancements to make our lives better. With the help of research in the area of computer science, we can make devices like a smartwatch, pacemakers, and other devices which help mankind to have a better life. Directly designing the hardware, testing and implementing these hardware devices will cost us more. To make it cost-effective and instead of directly designing on hardware we use simulators to check for faulty designs, errors using FPGA and HDLs even before they are fabricated. FPGAs evolved from Programmable Read Only Memory and Programmable Logic Array. It is a hardware circuit, programmed by the user to carry out logical operations. FPGA is a collection of programmable logic gates. These arrays of PLA can be programmed to behave like a logical operation or memory. Standard embedded microprocessor and microcontroller are designed and fabricated. We write the software for these ICs, which later can be upgraded or erased. With FPGA, we are designing the hardware circuits. Here we write the functionality of the hardware which can be as simple as basic logic functions or as complex as a co-processor. The real purpose of the FPGA is to have optimized application-specific ICs. It is a perfect choice for the prototyping model because

of the reprogramming feature. This helps the user to test and make it bug-free before the actual manufacturing. Hardware manufacturers chooses the FPGA technology for prototyping its new chip design. Having an optimized FPGA gives the upper hand in terms of versatility, performance, power efficiency, and performance. To code the hardware, we need a programming language. We have C, C++, Java, and Hardware description language(HDL). But to write the hardware design and dump it onto the FPGA we need HDL. We specifically need HDL because it is concurrent in nature and other languages are sequential. With a sequential approach, we can never aim for high-performance computing. Therefore, the apt programming language for FPGA is HDL.

II. FPGA ARCHITECTURE

Some chip/hardware manufacturers like Intel, Xilinx, Lattice owns FPGA architecture. Let us walk through the general architecture of FPGA. Every FPGA has a Programmable Logic element and Programmable interconnect structure. The Fig. 1 depicts the fabric of FPGA[1].

A. The FPGA architecture consists of

- Configurable Logic blocks
- Configurable I/O blocks
- Interconnecting Wires
- Switch Matrix

1) *Configurable Logic Block [2]*: The heart of FPGA is Configurable Logic Block(CLB). FPGA architecture is an array of CLBs. These logical modules are like blank paper. We can develop and test any circuit. These blank logical modules allow the user to configure the digital circuit, test, and debug. FPGA allows the user to configure and reconfigure the blocks n number of times. Here each CLB is customized to carry out specific computing. Fig.2 depicts CLB and their interconnection.

2) *Configurable I/O Block*: This allows the CLB to interact with the external environment. I/O block helps to bring in the signals and send out the signals from the FPGA.

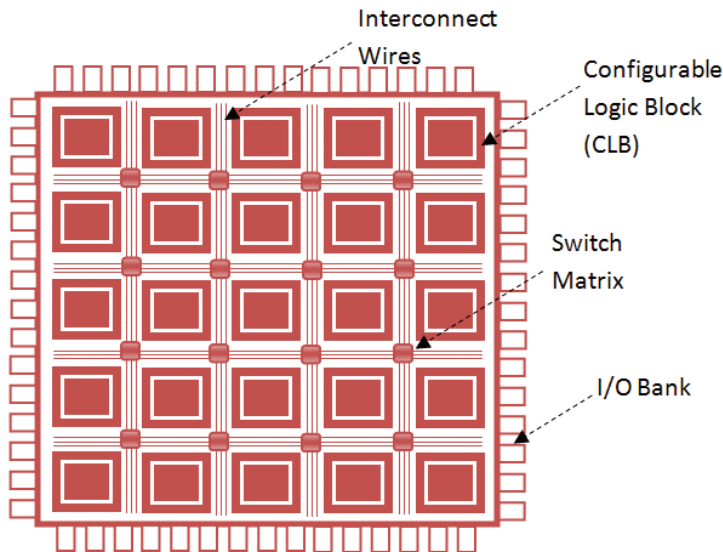


Fig. 1. FPGA Architecture

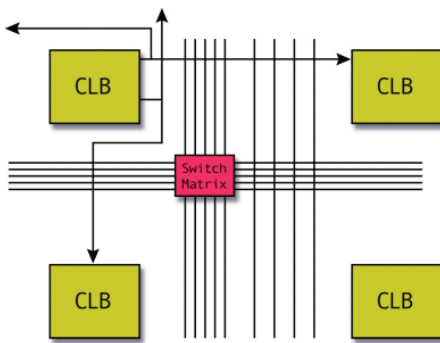


Fig. 2. Configurable Logic Block

3) *Interconnecting Wires and Switch Matrix*: This resource allows all the CLBs to connect via switch matrix to implement the complex design.

In FPGA, CLBs are arranged in a two-dimensional structure and a connection path has to be made between the CLBs and among the wires themselves. The routing channels help the wires route to the correct path. As an FPGA architect, you have the leverage to choose the interconnecting wires. FPGA channels provide varying length wires because CLBs are arranged regularly in a structured manner.

III. PECULIARITY OF FPGA

The peculiarity of the FPGA is we write the software code, to develop the hardware which may be the IC to do any specific task. We write the code and check for the functionality and timing constraints even before we think of fabrication. FPGA is the most cost-effective way to design hardware circuits and check their functionality, timing, and behavioral design. Before using, the FPGAs need to be configured. In this paper, we will understand how FPGA is programmed and what kind

of leverage the FPGA gives to the designer before going for fabrication. This one such boon from the technological perspective has helped human mankind to make the chips without any huge cost involved. Unlike other embedded microprocessors/microcontrollers, whose configuration is sealed by the manufacturer and cannot be changed.

IV. APPLICATIONS OF FPGA

The FPGA is a preferred solution in many applications because it can be configured, reconfigured as needed, and tailored for a specific set of functions. Most of these application areas are undergoing rapid change as new protocols and standards are implemented and changes constantly. Manufacturers can implement systems that can be changed as needed thanks to FPGAs. Specific application of FPGA include

- Medical Imaging
- FPGA Prototyping
- Digital Signal Processing
- Bioinformatics
- Voice Recognition
- Cryptography

A. Medical Imaging

Systems for diagnostic imaging, such X-ray and ultrasound, have been in use for decades. Newer systems include nuclear or positron emission tomography (PET), computed tomography (CT), and magnetic resonance imaging (MRI). Due to the complexity and image processing requirements of these modern diagnostic imaging systems, manufacturers are constantly introducing new features and enhancing performance. Having FPGA, we can leverage the opportunity to upgrade the module as and when the research is implemented.

B. FPGA Prototyping

FPGA prototyping is a necessary tool to handle system validation and software execution since it operates at a rate that is roughly an order of magnitude quicker than emulation for the same design size. The faster execution speed is attained by sacrificing the DUT debugging capabilities and quick compilation times, which are necessary for quick design iterations but not crucial for prototype applications.

The functionality of a new design can also be demonstrated via FPGA prototyping before silicon is available.

Due to the popularity of commercial FPGA prototyping services, industry competition resulted in mergers and acquisitions. Today's emulation providers also provide FPGA prototyping platforms, either through in-house development or through partnerships with FPGA prototyping companies.

C. Digital Signal Processing

FPGAs offer a reprogrammable solution that satisfies the needs of many DSP applications. Without in-depth technical knowledge of hardware description language, developers can build flow paired with hardware acceleration by using legacy codes to accomplish functions.

Additionally, FPGA and SoC development boards give designers scalable capabilities for quickly prototyping embedded

systems like DSPs. In comparison to the typical microprocessor, end-users have more design options when using a development board.

D. Bioinformatics

An extremely appealing alternative to software-only methods is to design dedicated hardware for bioinformatic algorithms. The hardware will be configured to run a certain algorithm, which will result in significant performance gains. FPGAs (Field Programmable Gate Arrays) are the obvious choice for these systems due to their flexibility. FPGAs have been heavily utilised in recent years to construct specialised circuits, particularly those intended to speed up computationally demanding Bioinformatics applications.

V. LANGUAGES FOR FPGA

The main purpose of using languages is to communicate. Here we use the Hardware Description Language (HDL) to interact with the FPGA by describing the hardware. HDL is a descriptive and simulative language for automatic hardware implementation. HDL languages/compiler allow the designer to describe the digital circuit textually or symbolically design the target hardware. The essential goal is to check the described target hardware to be error free, faulty designs and check for its performance.

VI. HDL APPROACH

During the evolution of HDL, the focal point was to design the hardware from gate level to register transfer to architectural level. Successive transitions led us to the behavioral level and structural level. Modeling the hardware at the gate level included instantiating the gates in hardware design, representing the gates using a Boolean expression, designing their interconnection blueprint, and identifying the propagation delay/ timing behavior. Hardware can be described at the behavioral, functional, and gate levels. Any HDL should allow Generic Modelling and creating an instance of the same. The circuits' logical behavior and timings should be indicated and precisely reflected. Interaction among the model/instance should be allowed. A hierarchical approach can be used to represent the design.

VII. PECULIARITY OF HDL

HDL programming allows the designer to design the hardware architecture. HDL approach is concurrent. For this reason, this becomes very important to focus on the timing aspect to get the desired functional behavior. Timing of the circuits includes propagation delay, path delay, clock trigger, and much more timing aspects we need to look into to make the circuit behave, the way it has to behave. This is the peculiar approach of HDL. Unlike the Programming Languages approach is uncomplicated sequential execution of programs and does not have a grip on lower-level details and their timings.

VIII. HARDWARE DESCRIPTION LANGUAGES V/S PROGRAMMING LANGUAGES

Programming Languages allows you to write the instruction to execute on the pre-designed hardware. Using a programming language, we can exploit the existing architecture by writing optimized code. As the machine cannot understand the programming language written in a high-level language, this needs a translator/compiler to convert the high-level language to machine code. This means along with the programming language we need additional software components to convert the high-level language to machine code. The nature of the execution is sequential. Therefore, we cannot bargain on space complexity and time complexity.

A. Hardware Description Languages

Digital designers always prefer to visualise their designs using HDL logic. They can achieve it, if and only if the hardware function is simple. As the complexity of the hardware increases, it becomes problematic to put across the manual design. This scenario made the digital designer rely more on a text-like language. This means to facilitate the hardware design which was complex to design, hardware description languages [3] were developed to make the life of the digital designer easy.

In HDL, we describe the hardware circuit in a normal text narrative format. HDL tools permit automatic hardware implementation using the software. This also helps us to define the structure and behavior of the circuit. Hardware circuits could be defined from abstract level to gate level to transistor level. We simulate the hardware circuit to test for faulty design even before fabrication. This makes hardware design very cost-effective to build new ones. The most time-consuming part of HDL is writing code and testing. Once the code is ready and it is dumped on FPGA, only important factor we need to consider is time. Because, unlike programming languages, HDL is not sequential but its nature of execution is concurrent, hence faster in execution compared to the other sequential programming languages.

Timing, particularly relative timing, which emphasizes the occurrence of all hardware actions or events, is a critical component in digital hardware design. Timing determines the accuracy of and directly impacts the effectiveness of the digital design

Early HDLs, attracted the attention of the designers because the HDL allowed the designers to construct logic gates using the syntax of the language. The simulators helped to check the output of the circuit. The growth and use of a large number of gates caused a struggle to comprehend an unhurried simulation speed. To overcome these issues, advanced features in register transfer level languages with computer-aided design, digital system design language, and finally Hardware Programming language along with their simulators are used in designing digital circuits. With the advent of these tools, abstract designs were built.

As the level of abstraction increased, the instruction set level was introduced to model computer architectures. This is the highest level of abstraction where all the lower-level implementation details are hidden. Efficiency is based on the instruction set, traffic between registers, memory, and other elements. Instruction Set Processing models are synchronous computer design. The main executable program fetches the instructions, decodes them, and then executes each one at a time.

IX. FPGA IMPLEMENTATION

The following is the flowchart for FPGA Implementation [4]

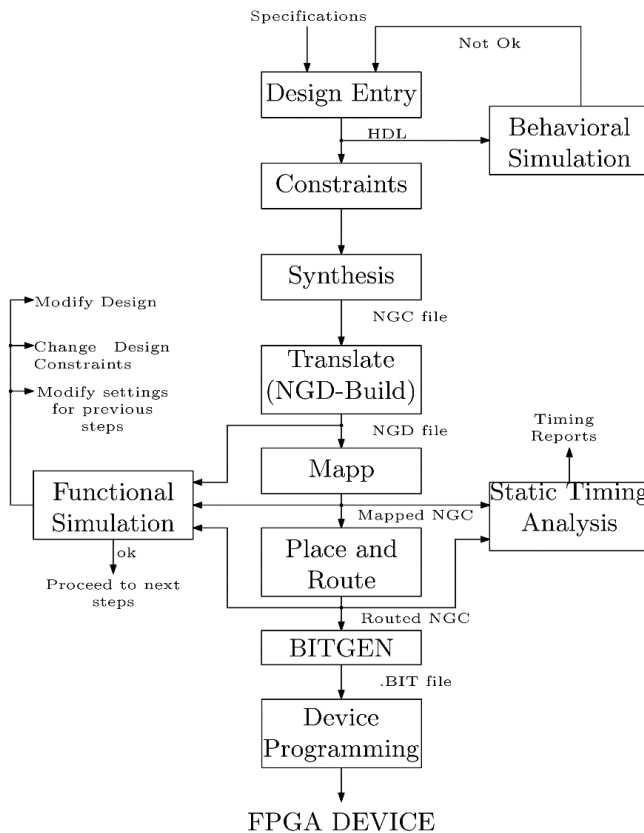


Fig. 3. Flowchart for FPGA Implementation

Specification: Here we specify the requirements to build the circuit. The requirement is specified the input and output of the digital circuit design. It should also tell how the input is converted as output.

A. Design Entry

The following are the different techniques for design entry

- Schematic
- HDL – Hardware description language
- State Machine

The method of choice is influenced by the design and the designer. Schematic entry is a better choice if the designer wishes to work with hardware more. HDL is a

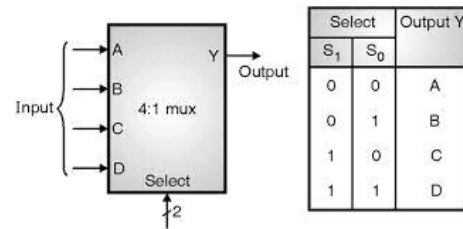


Fig. 4. 4-1 Multiplexer Requirement

preferable option when the design is complex or the designer wishes to implement the design algorithmically. Although language-based entry is simpler, it performs and has a lower density. The level of abstraction offered by HDLs can shield designers from the specifics of hardware implementation. Designers have significantly better visibility into the hardware with schematic-based entry. State machines are another approach; however, they are rarely employed. For designers that view the design process as a sequence of states, it is the best option. But there are few resources available for state machine entry. Schematic and HDL are both used in the structural style of design entry. To comprehend hardware specifics without a lot of complexity, we preferred design entry through VHDL/Verilog HDL in structural style.

B. Synthesis

The process of converting VHDL or Verilog code into a device netlist format, or a full circuit with logical components (gates, flip-flops, etc.), for the design, is known as synthesis. If the design consists of multiple sub-designs, for instance, a processor must be implemented, a CPU must be one design element, RAM another, and so forth, the synthesis process will produce a netlist for each design element. The synthesis process verifies that the design is optimized for the design architecture that the designer has decided by verifying the code syntax and analysing the design hierarchy.

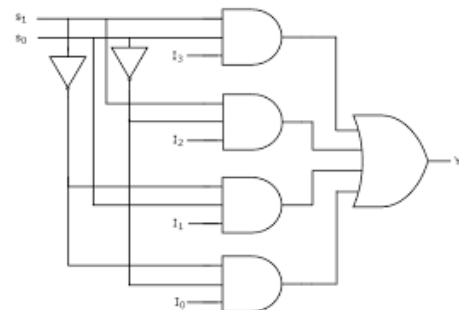


Fig. 5. 4-1 Multiplexer Synthesised Logic Diagram

C. Implementation

This consist of 3 steps in the following order

- Translate

- Map
- Place and Route

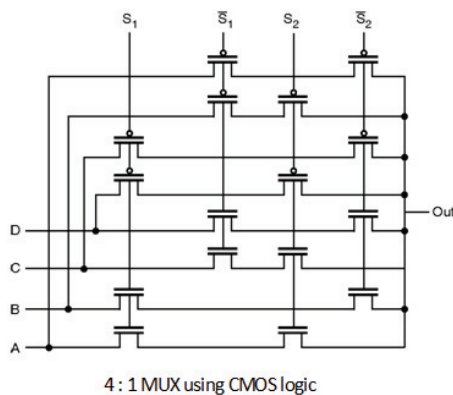


Fig. 6. 4-1 Multiplexer Transistor Level Logic Diagram

```

module mux4x1_bh (i0,i1,i2,i3,select,y);
  input i0,i1,i2,i3;
  input [1:0] select;
  output y;
  reg y;
  always @ (i0 or i1 or i2 or i3 or select)
    case (select)
      2'b00: y = i0;
      2'b01: y = i1;
      2'b10: y = i2;
      2'b11: y = i3;
    endcase
endmodule

```

Fig. 7. 4-1 Multiplexer Textual Format (Verilog Code)

Step 1: Translate - All of the input netlists and restrictions are combined throughout the translation process to create a logic design file. An NGD (Native Generic Database) file is used to store this data. Utilizing the NGD Build software, this is possible. By assigning ports in the design to physical components (such as pins, switches, buttons, etc.) of the intended device and stating the design's time restrictions, constraints are defined in this context. This data is kept in a file with the suffix UCF (User Constraints File).

Step 2: Map - The entire circuit with logical components is divided into smaller blocks by the mapping process so that they can fit into the logic blocks of an FPGA. By fitting the logic specified by the NGD file into the desired FPGA elements (such as the Combinational Logic Blocks (CLB) and Input-Output Blocks (IOB)), the mapping procedure creates an NCD (Native Circuit Description) file that figuratively depicts the design mapped to the FPGA components. This is achieved using the MAP program. All the information about the physical constraints is also generated into a PCF (Physical Constraints File).

Step 3: Place and Route: This procedure makes use of the Place and Route - PAR program. The place and route

process joins the logic blocks and inserts the subblocks from the mapping process into the logic blocks in compliance with the constraints. For instance, if a subblock is positioned in a logic block that is very close to an IO pin, it may reduce time but may also result in other constraints. The place and route procedure thus consider a trade-off between all the limitations. The output of the PAR tool, which accepts the mapped NCD file as input, is a fully routed NCD file. The routing data is contained in the output NCD file.

Multiple optimization techniques are conducted during the MAP process to trim or remove redundant, unnecessary, and irrelevant logic parts. For smaller designs, the optimum frequency obtained after the synthesis process and after the MAP process can be matched. However, for complicated designs, the frequency obtained using the MAP process is typically lower than the frequency obtained by the synthesis process. The maximum frequency can be calculated using the following formula:

Total Delay = Data Path Delay (Logic Delay) + Clock Path Skew + Clock Uncertainty + Routing Path Delay

D. Device Programming

The FPGA now needs to have the design loaded. But, for the FPGA to accept the design, it needs to be converted into a format. Conversion is handled by BITGEN software. To create a bit stream that may be used to configure the target FPGA device, the routed NCD file is then passed to the BITGEN program. Use of a cable is possible here. Design influences the choice of cable.

X. DESIGN VERIFICATION

Design Verification [6] is done at every stage. Following are the different simulation processes.

A. Behavioral Simulation

The structure of the design flow hierarchy includes several levels that involve simulation; this is the first of such steps. This simulation is run before the synthesis process to validate the RTL (behavioral) code and make sure the design is functioning as intended. Verilog or VHDL designs can both be subjected to behavioral simulation. Signals and variables are observed during this process, along with the monitoring of processes and functions and the setting of breakpoints. Due to the speed of this simulation, the designer can modify the HDL code if the needed functionality cannot be achieved in a short amount of time. Timing and resource use properties are still unclear because the design hasn't been synthesized to the gate level yet.

B. Functional Simulation

Functional simulation provides detailed information about the circuit's logic behavior. After the translate process, the

designer can use this method to check the functionality of the design. If the functionality is not what was predicted, the designer must make code adjustments and repeat the design flow processes, time frame. Timing and resource utilization characteristics have not yet been determined because the design has not yet been synthesized to the gate level. Functional simulation may succeed in some cases, although designs aimed at FPGA devices may fail to deliver the desired results. This is a result of the PAR process' strict optimization and lack of compliance with the limitations. To test the functionality following PAR, a post-PAR simulation can be run. It is anticipated that the design can be effectively executed if the post-PAR simulation is satisfactory.

C. Static Timing Analysis

After the MAP or PAR processes, this can be done. Signal route delays of the design determined from the design logic are listed in the post MAP timing report. A thorough timing summary of the design is provided by the post place and route timing report, which combines timing delay information.

XI. COMPUTATIONAL ANALYSIS

We analysed to find which programming methodology is efficient in terms of time complexity. The languages chosen are C, C with OpenMP, and HDL using Verilog. A sample experiment taken is nxn matrix multiplication.

The time complexity for nxn matrix multiplication in single core architecture with a single thread is $O(n^3)$ [5]. The implementation is done using the C language on the Linux platform. This takes more time compared to the other two methods because the execution of the C program is sequential and is executed on a single core with a single thread pattern. Now let us consider, the time complexity of the same nxn matrix multiplication using a dual-core machine with two threads per core. This is done using OpenMP construct using C language in the Linux platform and the order of complexity is $O(n^3)/4$. Here four indicates the number of threads executing in parallel using a dual-core with two threads per core.

Consider, the same logic executed using HDL. This is done using Verilog and the time complexity is $O(n^3)/m$ where m is the concurrent instance at a given time.

Therefore, the time taken by HDL is much lesser compared to the other 2 execution platforms.

XII. CONCLUSION

Regarding analysis, we conclude that HDL takes much lesser time compared to the other two methodologies that is embedded microprocessor and microcontroller. The limiting factor for the microcontroller is the clock speed and its inability to execute the code in parallel. Microprocessor which is competent enough to run the multiple threads when microprocessor has more than a single core. The only limiting

factor for a microprocessor is its clock speed and the number of thread it can execute in parallel. Here certain degree of parallelism is achieved. To overcome this limiting factor we can use HDL, where parallelism is achieved to the highest extent.

REFERENCES

- [1] <https://allaboutfpga.com/fpga-architecture/>
- [2] <https://www.eetimes.com/all-about-fpgas/>
- [3] Sumit Ghosh, Hardware Description Languages Concepts and Principles, IEEE Press Series on Microelectronic System, Prentice-Hall India.
- [4] <https://digitalsystemdesign.in/fpga-implementation-step-by-step/>
- [5] <https://www2.seas.gwu.edu/narahari/cs211/materials/lectures/algorithms.pdf>
- [6] <https://hardwarebee.com/ultimate-guide-fpga-design-flow/>
- [7] J. P. Banatre, D. Lavenier and M. Vieillot, "From high level programming model to FPGA machines," Proceedings of IEEE Workshop on FPGA's for Custom Computing Machines, 1994, pp. 119-124, doi: 10.1109/FPGA.1994.315590.
- [8] Steve Kilts, Advanced FPGA Design: Architecture, Implementation, and Optimization, Wiley-IEEE Press, 2007
- [9] Ian Kuon; Russell Tessier; Jonathan Rose, FPGA Architecture: Survey and Challenges , now, 2008.